

LearnLens

Application Scope Vision and Development Plan

Field	Details
Course	PED 741 – Software Development Practice
Deliverable	Lab 1 Application Development Plan
Team members	Ansar Murat and Ardak Artykbayeva
Instructor	Nurbol Aidarbayev
Submission date	30.09.2026

PUBLIC GITHUB CLASSROOM REPOSITORY

<https://github.com/ansxxsar/learnlens>

Executive Summary

LearnLens is an AI-assisted formative assessment platform for introductory programming courses. It addresses the delay and repetition involved in reviewing large numbers of code submissions. The first release combines instructor-defined tests and rubrics with personalized AI explanations, revision opportunities, short reinforcement questions, and progress tracking. It is designed to support instructors rather than replace their academic judgment.

The MVP will deliver one complete Python learning cycle: create an assignment, submit code, execute it safely, generate evidence-grounded feedback, answer a short follow-up question, resubmit, and review progress. A responsive web application will support laptop and mobile browsers. Native mobile applications, multi-language execution, a complete LMS, and fully autonomous final grading are outside the first release.

1 Application Vision

1.1 Problem

Programming instructors frequently spend significant time checking similar errors and repeating feedback. Students may wait too long for guidance or receive only a score and failed-test list. This weakens the feedback loop that helps novices connect code behavior to programming concepts.

1.2 Vision Statement

LearnLens will help programming students improve through immediate, explainable feedback on each attempt while giving instructors control over criteria, test evidence, review, and final assessment decisions.

1.3 Value Proposition

User	Current difficulty	LearnLens value
Student	Delayed or unclear feedback	Specific strengths, issues, next steps, and revision history
Instructor	Repetitive first-pass review	Automated tests, draft feedback, override controls, and error analytics
Course team	Limited view of learning trends	Skill and misconception summaries across the semester

2 Objectives and Success Measures

Objective	Initial measure
Faster feedback	Median deterministic result returned within 30 seconds under normal load
Useful guidance	At least 80 percent of pilot users can identify a next action from the feedback
Lower repetitive workload	Instructor reports reduced time on common first-pass comments during the pilot
Learning progress	Dashboard shows improvement across attempts and mastery by skill
Instructor control	Every automated result can be reviewed and overridden with an audit record

3 Scope

3.1 MVP Features

- Student and instructor accounts with course-based authorization.
- Python assignments with instructions, deadlines, rubrics, public tests, and hidden tests.
- Immutable submission attempts and status tracking.
- Isolated code execution with strict resource and network limits.
- Personalized feedback grounded in rubric criteria and test outcomes.
- Instructor review and override of automated assessment output.
- Short post-submission reinforcement questions and explanations.
- Student progress with skills, XP, levels, and a small set of achievements.
- Instructor dashboard for completion and common-error patterns.

3.2 Post MVP

- Additional programming languages and richer IDE features.
- Native Android and iOS applications.
- LMS integrations such as Moodle grade and roster synchronization.
- Adaptive learning paths, social challenges, and advanced badges.
- Advanced plagiarism investigation workflows.

3.3 Explicit Non Goals

LearnLens will not become a complete LMS in the MVP, automatically make final misconduct decisions, expose complete solutions by default, or use an LLM as the sole basis for official grades.

4 Users and Core Scenarios

Role	Core scenario	Expected outcome
Student	Submit code and inspect feedback	Understands what worked, what failed, and what to try next
Student	Revise and resubmit	Can compare attempts and see demonstrated progress
Instructor	Publish an assessment	Controls instructions, rubric, tests, timing, and feedback depth
Instructor	Review class performance	Finds common misconceptions and students who may need support
Administrator	Monitor the service	Maintains access, reliability, configuration, and auditability

5 Functional Requirements

- The system shall authenticate users and enforce course-level roles.
- Instructors shall create, publish, close, and archive Python assignments.
- Students shall create immutable code-submission attempts and see processing state.
- The platform shall run untrusted code only in isolated workers.
- Deterministic tests shall finish before AI feedback is requested.

- Feedback shall reference approved rubric and execution evidence.
- The system shall provide a deterministic fallback when AI output is unavailable or invalid.
- Instructors shall inspect and override automated results.
- Students shall receive a concept-linked reinforcement question after a completed attempt.
- Dashboards shall show student progress and authorized course-level summaries.

6 Quality Security and Ethics

Area	Requirement
Security	No untrusted code in the API process; isolate execution and block outbound network access
Privacy	Collect necessary data only and restrict access through course membership and role
Reliability	AI failure must not hide deterministic results or previous attempts
Accessibility	Target WCAG 2.2 AA for core keyboard and screen-reader flows
Explainability	Trace feedback to a rubric criterion, test result, or code location
Integrity	Avoid complete solutions and separate practice rewards from official grades
Auditability	Record prompt, model, evidence version, override, and assessment-relevant changes

7 Solution Design

The responsive React and TypeScript client will communicate with a Django REST API. PostgreSQL will store course and assessment records. Celery and Redis will run long tasks asynchronously. Disposable Docker workers will execute Python submissions. The AI feedback service will receive a constrained evidence package and return validated structured feedback. This separation prevents code execution and external model calls from blocking or compromising the main application.

Component	Responsibility
React client	Student and instructor workflows across laptop and mobile browsers
Django API	Authentication, authorization, assignments, submissions, progress, and audit rules
PostgreSQL	Durable relational records and immutable attempt history
Celery and Redis	Execution and feedback job queues with status tracking
Execution worker	Restricted Python execution and structured test results
AI feedback service	Rubric-aligned explanation with schema validation and fallback

8 Development Roadmap

Milestone	Main deliverable
1 Foundation	Approved scope, repository documents, wireframes, data model, tooling, and CI

Milestone	Main deliverable
2 Course setup	Authentication, roles, courses, assignments, rubrics, tests, and deadlines
3 Submission	Immutable attempts, isolated runner, structured results, and failure handling
4 Feedback	Grounded AI feedback, fallback, instructor review, questions, and resubmission
5 Progress	Skill mastery, XP, levels, achievements, and cohort analytics
6 Release	Usability, accessibility, security, performance, documentation, and demonstration

9 Team and Repository Workflow

The two-person team will divide ownership by feature while reviewing each other's changes. GitHub Issues will contain acceptance criteria. Short-lived feature branches will be merged through pull requests after tests and review. The main branch will remain deployable.

- Use Conventional Commits such as feat:, fix:, docs:, test:, refactor:, chore:, and ci:.
- Keep pull requests small, link them to issues, and document assumptions and test evidence.
- Protect main and require passing automated checks when repository settings permit.
- Store environment examples without credentials and never commit student data or secrets.
- Keep README.md, specifications.md, development.md, and agents.md current.

10 Validation Plan

Validation will combine automated tests and small task-based user sessions. Backend tests will cover permissions and business rules; integration tests will cover the submission lifecycle; security tests will challenge timeouts, filesystem access, process creation, and network restrictions; contract tests will validate AI output and fallback behavior; and end-to-end tests will cover the instructor publish and student submit paths. Pilot participants will complete representative tasks and explain whether the feedback gives them a clear next step.

11 Risks and Mitigation

Risk	Mitigation
Unsafe execution	Disposable isolation, strict resource limits, no network, no host credentials, and hostile-input tests
Incorrect AI feedback	Evidence grounding, structured schema, confidence handling, fallback, audit record, and instructor override
Scope expansion	Explicit non-goals and completion of the vertical slice before optional features
Privacy or bias	Data minimization, authorization, retention policy, no public rankings, and human review
Academic integrity	No full solutions by default, attempt history, configurable guidance, and separate final judgment
Service cost or outage	Asynchronous calls, budgets and limits, monitoring, and preserved deterministic results

12 Lab 1 Completion Checklist

- Accept the GitHub Classroom assignment and confirm the repository URL.
- Set repository visibility to public if required by the course and verify it in a signed-out browser.
- Replace all bracketed cover-page fields in this document.
- Paste the live repository URL on the cover page.
- Place README.md, specifications.md, development.md, and agents.md in the main branch root.
- Create several meaningful Conventional Commits rather than one bulk upload.
- Upload this plan as DOCX or PDF to the course portal.
- Submit the direct repository URL in the assignment response.